

TRIN-FOR-TRIN

Pristilbud i Blazor

— fra tom mappe til hostet app

Sådan bygger du et Blazor WebAssembly-projekt fra prototypen: først checklisten alene, så et app-skelet med sidemenu, en tabel over sendte pristilbud og trin-flowet der laver et nyt — og til sidst en hurtig Release-version du kan dele.

Hvert trin i denne guide er udført og verificeret på den kode der ligger i `proto/BlazorProto` (commit `fddb995`). Følger du dem i rækkefølge, ender du med det samme resultat — og det færdige projekt ligger som reference i `OnboardingChecklist/` i repoet.

new-design-project · september 2026 · .NET 8

0 Før du starter

- **.NET 8 SDK.** Tjek med `dotnet --version` — du skal se `8.x`. Guiden er testet på 8.0.130.
- **Kildekoden.** Vi kopierer filer fra den eksisterende prototype. I kommandoerne nedenfor står `$SRC` for stien til den:

```
SRC=~/.dev/emil-kowalski/new-design-project/proto/BlazorProto
```

- **En terminal.** Alt her er kommandolinje + en teksteditor. Ingen Visual Studio nødvendig.

Hvorfor et nyt projekt og ikke bare slette de to andre varianter? Fordi prototype-harneset (picker, tastaturnavigation, variant-skift, `?v=N` i URL'en) er filtret ind i `Proto.razor`, `Picker.razor`, `picker.js` og en tredjedel af CSS'en. Det er hurtigere og renere at starte fra nul og hente præcis de filer checklisten bruger.

1 Byg projektet

1 Opret et tomt Blazor WASM-projekt

```
dotnet new blazorwasm -e -o OnboardingChecklist
cd OnboardingChecklist
```

`-e` betyder *empty*: ingen Bootstrap, ingen eksempelsider. Templaten laver stadig en router, et layout og en Home-side, som vi ikke skal bruge — slet dem sammen med dens CSS og index.html:

```
rm -rf App.razor Pages Layout wwwroot/css wwwroot/index.html
mkdir -p Model Components wwwroot/css wwwroot/js
```

Appen er én side uden navigation, så en `Router` og et `MainLayout` er kun støj. Root-komponenten monteres direkte i `Program.cs` (trin 3).

2 Slå kulturdata fra i .csproj

Åbn `OnboardingChecklist.csproj` og tilføj to linjer i `<PropertyGroup>`, lige efter `<ImplicitUsings>`:

```
OnboardingChecklist.csproj
<PropertyGroup>
  <TargetFramework>net8.0</TargetFramework>
  <Nullable>enable</Nullable>
  <ImplicitUsings>enable</ImplicitUsings>
  <InvariantGlobalization>true</InvariantGlobalization>
  <BlazorEnableTimeZoneSupport>false</BlazorEnableTimeZoneSupport>
</PropertyGroup>
```

Appen formaterer ingen datoer eller tal efter kultur. Uden de to linjer sender Blazor ~2,5 MB ICU- og tidszonedata med i hvert load, som aldrig bliver brugt.

3 Erstat Program.cs

```
Program.cs

using Microsoft.AspNetCore.Components.WebAssembly.Hosting;
using OnboardingChecklist;
using OnboardingChecklist.Model;

var builder = WebAssemblyHostBuilder.CreateDefault(args);

// Single page, no router: Onboarding is the root component.
builder.RootComponents.Add<Onboarding>("#app");

// Singleton so answers survive re-renders.
builder.Services.AddSingleton<OnboardingState>();

await builder.Build().RunAsync();
```

4 Erstat _Imports.razor

```
_Imports.razor

@using Microsoft.AspNetCore.Components.Web
@using Microsoft.JSInterop
@using OnboardingChecklist
@using OnboardingChecklist.Model
@using OnboardingChecklist.Components
```

Filen gælder for alle `.razor`-filer i projektet, så komponenterne kan referere til `OnboardingState`, `Content`, `Icons` osv. uden egne `@using`-linjer.

5 Kopiér datamodellen

```
cp $SRC/Model/Content.cs $SRC/Model/OnboardingState.cs Model/
```

I begge filer: skift første linje fra `namespace BlazorProto.Model;` til `namespace OnboardingChecklist.Model;`.

I `Model/OnboardingState.cs` skal de felter der kun fandtes for pickeren og de to andre varianter væk. Find blokken *picker + per-variant navigation* og slet de markerede linjer:

Model/OnboardingState.cs

```
public int ActiveVariant { get; set; }
public int MountCounter { get; set; }

public int StepperIndex { get; set; }
public bool StepperDone { get; set; }

public int ConvoIndex { get; set; }
public bool ConvoDone { get; set; }

public string ChecklistKey { get; set; } = "name";
public bool ChecklistDone { get; set; }
```

Og nederst i `ResetFlow()`:

```
public void ResetFlow()
{
    Errors.Clear();
    StepperIndex = 0; StepperDone = false;
    ConvoIndex = 0; ConvoDone = false;
    ChecklistKey = "name"; ChecklistDone = false;
}
```

`Content.cs` er alt indholdet — de fire opgaver, rollelisten, notifikationsvalgene, workspace-navnet. `OnboardingState.cs` er svarene, valideringen og de afledte værdier (hvor mange er færdige, hvad mangler). Begge er ren C# uden Blazor-afhængigheder, så de flytter med uændret.

6 Kopiér de delte komponenter

```
cp $SRC/Components/Icons.cs $SRC/Components/Recap.razor $SRC/Components/Fields.razor
Components/
```

- `Icons.cs`: skift namespace til `OnboardingChecklist.Components`.
- `Recap.razor`: ingen ændringer.
- `Fields.razor`: én ændring. Feltfokus går gennem et lille JS-kald, og funktionen skifter navn fra picker-objektet til vores eget:

Components/Fields.razor

```
public ValueTask FocusFirstAsync() => JS.InvokeVoidAsync("protoPicker.focusFirst", root);
public ValueTask FocusFirstAsync() => JS.InvokeVoidAsync("app.focusFirst", root);
```

`Fields` renderer felterne for én opgave ad gangen (navn, rolle, invitationer, notifikationer) og var delt mellem Stepper og Checklist. Kommentaren øverst i filen om Stepper kan du slette — den passer ikke længere.

7 Kopiér checklisten som root-komponent

```
cp $SRC/Variants/Checklist.razor Onboarding.razor
```

Ingen ændringer i indholdet. Filnavnet bliver klassenavnet, så `Onboarding.razor` giver komponenten `Onboarding` — den som `Program.cs` monterer i `#app`.

8 Lav `wwwroot`

JavaScript. Én funktion, otte linjer — resten af `picker.js` var harness:

```
wwwroot/js/app.js

// Blazor's FocusAsync() needs an ElementReference per field; one helper that
// finds the first input in a container is far less ceremony.
window.app = {
  focusFirst(container) {
    if (!container) return;
    const el = container.querySelector('input:not([type=hidden]), textarea');
    (el || container).focus();
  },
};
```

index.html. Skriv filen sådan (boot-baren udfyldes af Blazor via `--blazor-load-percentage`, og fejl-elementet er skjult indtil der sker en rigtig fejl — begge dele styles i CSS'en):

```
wwwroot/index.html

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Onboarding</title>
  <base href="/" />
  <link rel="stylesheet" href="css/app.css" />
  <link rel="icon" type="image/png" href="favicon.png" />
</head>
<body>
  <div id="app">
    <div class="boot" role="status" aria-label="Loading">
      <div class="boot-bar"><span class="boot-fill"></span></div>
    </div>

    <div id="blazor-error-ui">
      An unhandled error has occurred.
      <a href="" class="reload">Reload</a>
      <a class="dismiss"></a>
    </div>

    <script src="js/app.js"></script>
    <script src="_framework/blazor.webassembly.js"></script>
  </body>
</html>
```

Kopier også `favicon.png` fra `$SRC/wwwroot/`.

CSS. `$SRC/wwwroot/css/app.css` er delt op i fem sektioner med tydelige bannere (`/* ==== NAVN ====` `*/`). Lav en ny `wwwroot/css/app.css` med disse tre dele, i denne rækkefølge:

TAG MED	FRA ... TIL	HVORFOR
MATERIALS	Banneret <code>MATERIALS</code> til og med <code>@media (forced-colors: active) { ... }</code>	Tokens, base, knapper, felter, boot-bar, fejl-bjælke
Tick + recap	Fra <code>.v1-tick {</code> til og med <code>.recap span { ... }</code> (inkl. <code>@keyframes pop</code>)	Bruges på "You're all set"-skærmen
VARIANT 3 — CHECKLIST	Banneret til filens slutning	Selve checklisten

Udelad: `PICKER` (øverst), resten af `VARIANT 1 — STEPPER` og hele `VARIANT 2 — CONVERSATIONAL` . Resultatet er ca. 530 linjer mod 934.

Klassenavnene `v3-*` og `v1-tick` stammer fra prototypen. De virker som de er; omdøb dem først når appen kører.

9 Kør den

dotnet run

Terminalen skriver `Now listening on: http://localhost:XXXX` — åbn den adresse. Du skal se:

- ☐ En tynd lilla bar i midten under load, som fylder op (ikke tekst)
- ☐ "Finish setting up" med rail til venstre (2 of 4 complete, 50 %) og panelet "What's your name?" til højre
- ☐ Klik **Save and continue** → panelet skifter til "What do you do?"
- ☐ Vælg en rolle, gem, klik **Finish setup** → "You're all set, Bart." med recap
- ☐ Ingen tekst nederst på siden, ingen scrollbar

Får du gammel visning efter en ændring: hard reload (`Ctrl+Shift+R`). Blazor cacher runtime-filerne aggressivt.

10 Hvis noget fejler

SYMPTOM	ÅRSAG	LØSNING
The type or namespace name 'BlazorProto' could not be found	Et namespace blev ikke skiftet	Søg efter <code>BlazorProto</code> i alle filer; skift til <code>OnboardingChecklist</code>
'OnboardingState' does not contain a definition for 'StepperIndex'	Du slettede et felt der stadig bruges	Du har kopieret en anden variant end <code>Checklist.razor</code>
Fejl i konsollen: Could not find 'protoPicker.focusFirst'	Trin 6, JS-navnet	Ret til <code>app.focusFirst</code> i <code>Fields.razor</code>
Appen loader men er ustylet	<code>index.html</code> peger på forkert CSS, eller CSS'en mangler <code>MATERIALS</code>	Tjek <code><link href="css/app.css"></code> og at <code>:root {</code> findes i filen
"An unhandled error has occurred" nederst, men appen virker	CSS-reglen for <code>#blazor-error-ui</code> mangler	Den ligger i <code>MATERIALS</code> — sektion ikke kopieret helt

2 Gør den til en to-sides app

Del 1 giver en app med én skærm. Herfra bliver den et rigtigt app-skelet: en sidemenu i venstre kant med to punkter — listen over sendte pristilbud, og trin-flowet der laver et nyt.

Hvert trin herunder svarer til én commit i repoet, så du kan læse den præcise ændring med `git show <hash>` hvis noget er uklart.

1 Back og Next i panelets fod — e33c0ed

Railen lader dig åbne trinnene i vilkårlig rækkefølge. Foden går dem igennem lineært. I

`Onboarding.razor` erstattes fodens indhold:

```
Onboarding.razor — .v3-panel-f
<div class="v3-panel-f">
  @if (Prev is { } prev)
  {
    <button type="button" class="btn btn-ghost" @onclick="()" =>
Open(prev.Key)">@Icons.ArrowL Back</button>
  }
  <div class="v3-panel-f-end">
    @* Skip og den primære knap ankres til højre, så primæren aldrig flytter sig *@
  </div>
</div>
```

Og i `@code`-blokken erstattes `NextIncomplete` med lineære naboer:

```
private int Index => Content.IndexOf(S.ChecklistKey);
private TaskDef? Prev => Index > 0 ? Content.Tasks[Index - 1] : null;
private TaskDef? Next => Index < Content.Tasks.Length - 1 ? Content.Tasks[Index + 1] : null;
```

Back er *skjult* på første trin, ikke deaktiveret. En knap der aldrig kan gøre noget hører ikke til på skærmen. På sidste trin bliver primæren "Save and finish" og kalder `Finish()`, som stadig går tilbage til et manglende krævet trin.

2 App-skelettet — 39cb774

Fire nye filer og en omrokering:

```
mkdir -p Pages
git mv Onboarding.razor Pages/NewQuote.razor
```

`Model/AppState.cs` — hvilken af de to sider der vises:

```

Model/AppState.cs
namespace OnboardingChecklist.Model;

public enum AppPage { Sent, Quote }

public class AppState
{
    public AppPage Page { get; set; } = AppPage.Sent;
}

```

`Model/Quote.cs` holder de sendte pristilbud. Bemærk indholdet: et meget langt klientnavn, en kladde uden beløb og dato, og en afvist række — tabellen skal overleve mere end det pæne tilfælde.

```

Model/Quote.cs
public enum QuoteStatus { Draft, Sent, Viewed, Accepted, Declined }

public record Quote(string Ref, string Client, QuoteStatus Status, decimal Amount, string Sent)
{
    public string Label => Status.ToString();

    public static readonly Quote[] All =
    [
        new("Q-2418", "Featherstonehaugh-Villanueva International Systems",
QuoteStatus.Viewed, 48_200m, "2026-09-02"),
        new("Q-2414", "Solberg Media", QuoteStatus.Draft, 0m, "-"),
        // ... seks til
    ];
}

```

`Shell.razor` i projektets rod bliver den nye rodkomponent: sidemenu til venstre, én side ad gangen til højre.

Ingen `Router`. To sider tjener ikke URL-routing hjem, og at indføre den ville betyde at hente `App.razor` og routeren tilbage — dem vi bevidst slettede i del 1, trin 1.

Endelig i `Program.cs`:

```

builder.RootComponents.Add<Onboarding>("#app");
builder.RootComponents.Add<Shell>("#app");
builder.Services.AddSingleton<OnboardingState>();
builder.Services.AddSingleton<AppState>();

```

Og `@using OnboardingChecklist.Pages` tilføjes i `_Imports.razor`.

Navnekollision: kald ikke siden `quote.razor`. `Quote` er allerede datamodellen, og `<Quote />` ville være tvetydig. Derfor `NewQuote.razor`.

`Pages/Sent.razor` er tabellen. Sortering og filtrering er LINQ — det som en tilsvarende vanilla-prototype bruger 90 linjer JavaScript på:

```
Func<Quote, IComparable> key = sort switch
{
    "ref" => r => r.Ref,
    "client" => r => r.Client,
    "amount" => r => r.Amount,
    _ => r => r.Sent,
};

var ordered = asc ? rows.OrderBy(key) : rows.OrderByDescending(key);
// En kladde har ingen dato. "-" sorterer som tekst over enhver rigtig dato
// faldende, så udaterede rækker skubbes bagest i begge retninger.
return ordered.OrderBy(r => r.Sent == "-" ? 1 : 0).ToArray();
```

3 Fælles indholdsbredde — 9d5146f

Tabellen fyldte alt hvad sidemenuen levnede: 2246 px ved et 2560 px vindue, med 1582 px mellem en rækkes reference og dens dato. Begge sider kappes nu ved samme token, så indholdets venstrekant ikke hopper når man skifter side.

```
wwwroot/css/app.css

:root {
    --page-w: 940px;
}

.v3-shell { max-width: var(--page-w); margin: 0 auto; }
/* border-box er global, så max-width dækker polstringen */
.page { max-width: calc(var(--page-w) + 64px); margin: 0 auto; padding: 44px 32px 64px; }
```

Tabellen får faste kolonner, så klientnavnet afkortes ved kolonnens kant i stedet for ved et vilkårligt `ch`-loft med tom plads ved siden af:

```
.t { table-layout: fixed; min-width: 660px; }
.t th:nth-child(1) { width: 118px; } /* Reference */
.t th:nth-child(3) { width: 116px; } /* Status */
.t th:nth-child(4) { width: 116px; } /* Amount */
.t th:nth-child(5) { width: 128px; } /* Sent */
.t-client { max-width: 34ch; }
.t-client { max-width: 100%; }
```

4 Trinnene ind i sidemenuen — 11f15d9

Da skallen gav siden en rigtig sidemenu, stod der pludselig to lodrette navigationssøjler 8 px fra hinanden: app-navigationen, og rail-kortet der efterlignede den. Railen flytter ind som indrykkede trin under sit eget menupunkt.

Opret `Components/StepRail.razor` med trinnene, og render den i `Shell.razor` :

```
Shell.razor
@if (App.Page == AppPage.Quote && !S.ChecklistDone)
{
    <StepRail />
}
```

Trinnene og felterne er nu ikke længere forælder og barn, så de kan ikke gentegne hinanden med `StateHasChanged`. Navigationen flytter op på den tilstand de deler:

```
Model/OnboardingState.cs
public event Action? Changed;
public void NotifyChanged() => Changed?.Invoke();

public string? Nudge { get; set; }
public bool FocusNext { get; set; }

public void Open(string key) { ChecklistKey = key; Errors.Clear(); FocusNext = true;
NotifyChanged(); }
public void Finish() { /* går til første manglende krævede trin, ellers ChecklistDone */ }
```

Begge komponenter abonnerer:

```
@implements IDisposable

protected override void OnInitialized() => S.Changed += StateHasChanged;
public void Dispose() => S.Changed -= StateHasChanged;
```

Fælden: Fields sender sin `Changed` videre til forælderen. Lader du den stå som `Changed="StateHasChanged"`, gentegnes kun panelet, og sidemenuens måler bliver hængende på det gamle tal. Den skal være `Changed="S.NotifyChanged"`.

Med railen væk fra indholdet kapper panelet sig selv, i stedet for at strække et navnefelt ud over 900 px:

```
.v3-single .v3-panel { max-width: 620px; }
```

5 Tilgængelighedsgulvet — 91f3bba

To målte fejl, begge blokerende.

Kontrast. `--ink-3` var `#a1a1aa` — **2,56:1** på hvid, langt under gulvet på 4,5:1 for brødtekst. Tokenet bruges udelukkende som tekstfarve, 19 steder, så det rettes ét sted:

```
--ink-3: #a1a1aa;
--ink-3: #67676f;
```

BAGGRUND	FØR	EFTER
hvid	2,56:1	5,61:1
--accent-soft	2,56:1	5,01:1
pillens #f0f0f1	2,56:1	4,92:1

Prisen er at den neutrale skala trykkes sammen mod --ink-2 (7,7:1). Det gamle spring var købt ved at falde under gulvet.

Tap targets. Gulvet er 44×44 px. Sidemenu, sorteringshoveder og felter havde plads til at vokse; knapperne beholder deres udseende og får gulvet gennem et hit-areal:

```
.side-nav-item { min-height: 44px; }
.f-input      { min-height: 44px; }
.t-th        { min-height: 44px; }

/* 38px er rigtigt for en desktop-knap; 38px hit-areal er ikke. */
.btn { position: relative; }
.btn::after {
  content: "";
  position: absolute;
  left: 0; right: 0; top: 50%;
  height: 100%;
  min-height: 44px;
  transform: translateY(-50%);
}
```

6 Bevægelse, navn og dødt CSS — c78be7d

- **Varigheder.** Målerens fyld 480 ms → **260 ms**, nudge 480 ms → **300 ms**. Gulvet er under 300 ms medmindre afstand eller kurve retfærdiggør mere.
- **Reduced motion.** Blokken var et fladt 0.01ms på alt, hvilket også dræber de opacity- og farveovergange man skal *beholde*. Nu falder bevægelsen væk og fades bliver.
- **Gruppenavn.** Trinlisten mistede sit navn da den forlod sin <aside aria-label="Setup checklist">. Den er nu role="group" aria-label="Quote steps".
- **Dødt CSS.** Flytningen efterlod 11 forældreløse klasser — .v3-rail, .v3-grid, .v3-task og resten, 43 linjer.

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-property: opacity, color, background-color, border-color, box-shadow
  !important;
    transition-duration: 120ms !important;
  }
  .side-step[data-nudge] { animation: none !important; }
  .v3-meter-bar i { transition: none !important; }
}
```

7 Progressen på kortet — 4557547, 3bcebb7

Sidemenuen sagde den samme kendsgerning fire gange i én 224 px kolonne: to af fire udfyldte prikker, teksten "2 of 4 done", tallet "50%", og baren. Det numeriske resumé flytter ud; prikkerne bliver og bærer trin-for-trin-tilstanden.

Første forsøg lagde det øverst til højre i headeren. Det var forkert — det flugtede med ingenting. Det ender direkte over kortet, i kortets egen bredde:

Pages/NewQuote.razor — inde i .v3-single, før panelet

```
<div class="v3-progress">
  <div class="v3-progress-top">
    <span class="v3-progress-n">@S.DoneCount of @Content.Tasks.Length done</span>
    <span class="v3-progress-pct">@Pct%</span>
  </div>
  <div class="v3-meter-bar" role="progressbar" aria-label="Setup progress"
    aria-valuemin="0" aria-valuemax="@Content.Tasks.Length" aria-
    valuenow="@S.DoneCount">
    <i style="transform:scaleX(@Fraction)"></i>
  </div>
</div>
```

```
.v3-progress { max-width: 620px; margin-bottom: 14px; }
```

Den ligger *uden for* panelet, ikke inde i det. Panelet er nøglet på det aktive trin og kører sin entré-animation forfra ved hvert skift — indeni ville en stabil størrelse glide ind på ny hver gang du klikker et trin.

Finish setup var samtidig det eneste indrammede felt i en søjle af rammeløse rækker. Den deler nu én erklæring med menupunkterne:

```
.side-nav-item,
.side-finish-btn {
  display: flex;
  align-items: center;
  gap: 9px;
  width: 100%;
  min-height: 44px;
  padding: 8px 10px;
  border: 0;
  border-radius: 8px;
  background: transparent;
  /* ... */
}
```

Den beholder sin lave vægt. Panelets Save-knap er den eneste fyldte primærknap på skærmen — én primær handling pr. skærmbillede.

8 Kør den

dotnet run

- ☐ Sidemenu i venstre kant med **Sent quotes** (med tallet 8) og **New quote**
- ☐ Sent quotes: tabel med fem sorterbare kolonner, søgning, og en tom tilstand med "Clear search"
- ☐ New quote: trinnene indrykket under menupunktet, progressbar over kortet i kortets bredde
- ☐ Klik et trin i menuen → panelet skifter; klik **Back** i panelet → menuen følger med
- ☐ Vælg en rolle → måleren går fra 2 of 4 til 3 of 4 *med det samme*
- ☐ **Finish setup** med manglende rolle → springer til "What do you do?" med rød besked

Ser du gammel styling? Blazors dev-server sender kun `ETag` og `Last-Modified` på `app.css` — ingen `Cache-Control`. Browseren genbruger sin kopi uden at spørge. Hard reload med `Ctrl+Shift+R`, eller slå **Disable cache** til i DevTools' Network-fane.

3 Publish og host

`dotnet run` er et Debug-build: 193 separate `.wasm`-filer, 8,5 MB, ingen trimming. Det er fint til udvikling og forkert til alt andet. En Release-publish trimmer klassebiblioteket ned til det der faktisk bruges og komprimerer det.

1 Publicér

```
dotnet publish -c Release -o publish
```

Tager 10–20 sekunder. Det der skal på nettet ligger i `publish/wwwroot/` — alt andet i `publish/` kan ignoreres.

	FILER	REQUESTS	OVERFØRT	LOAD (LOCALHOST)
Debug (<code>dotnet run</code>)	193 <code>wasm</code>	202	8,5 MB	1,1 s
Release publish, brotli	30 <code>wasm</code>	38	1,8 MB	0,5 s

Målt på prototypen med de to `csproj`-flag fra trin 2. Ved 4 Mbit tager Debug-versionen 19 s og publish-versionen ca. 4 s — forskellen er størst for dem der ikke sidder på din maskine.

Ved siden af hver `.wasm` ligger en `.br` (brotli) og en `.gz`. En server der kender dem, sender den komprimerede udgave og sparer 70 %. En server der ikke gør, sender `.wasm` ukomprimeret (ca. 5,5 MB) — det virker stadig.

2 Test det lokalt

```
python3 -m http.server 8080 -d publish/wwwroot
```

Åbn `http://localhost:8080`. Python-serveren sender ukomprimeret, men det er 38 requests i stedet for 202, så det er stadig markant hurtigere end `dotnet run`. Stop med `Ctrl+C`.

3 Læg det online

Netlify Drop — hurtigst når du bare vil vise det til nogen:

1. Gå til `app.netlify.com/drop`
2. Træk mappen `publish/wwwroot` ind i browservinduet
3. Du får en URL med det samme. Netlify komprimerer selv, så du får brotli-hastigheden uden opsætning.

GitHub Pages — hvis det skal ligge ved siden af koden:

Bemærk: GitHub Pages er kun tilgængeligt for private repos med GitHub Pro. Med en gratis konto skal repoet være offentligt.

1. Lav en tom fil `publish/wwwroot/.nojekyll`. Uden den ignorerer Pages mappen `_framework/`, fordi den starter med underscore.
2. Sitet bliver serveret fra `https://<bruger>.github.io/<repo>/`, så ret `<base href="/" />` i `publish/wwwroot/index.html` til `<base href="/<repo>/" />`.
3. Push indholdet af `publish/wwwroot` til en `gh-pages` -branch (eller en `docs/` -mappe på main), og peg repoets *Settings* → *Pages* på den.

GitHub Pages sender ikke `.br` -filerne selv, så load er ~5,5 MB ukomprimeret. Stadig 38 requests og trimmet — helt brugbart.

4 Valgfrit: mindre runtime

```
sudo dotnet workload install wasm-tools
```

Med denne workload relinker publish selve .NET-runtimen (`dotnet.native.wasm`, den største enkelte fil på 2,8 MB) og fjerner det appen ikke bruger. Du behøver ikke ændre noget i projektet — næste `dotnet publish` bruger den automatisk. Kræver sudo, fordi SDK'en ligger i `/usr/lib/dotnet`.

A Appendiks: filerne i det færdige projekt

```
OnboardingChecklist/
├─ OnboardingChecklist.csproj    ← D1 trin
2
├─ Program.cs                   ← D1 trin
3, D2 trin 2
├─ _Imports.razor               ← D1 trin
4, D2 trin 2
├─ Shell.razor                  ← D2 trin
2
├─ Model/
│   └─ AppState.cs              ← D2 trin
2
│   └─ Content.cs               ← D1 trin
5
│   └─ OnboardingState.cs        ← D1 trin
5, D2 trin 4
│   └─ Quote.cs                 ← D2 trin
2
├─ Pages/
│   └─ NewQuote.razor            ← D1 trin
7, omdøbt i D2
│   └─ Sent.razor                ← D2 trin
2
├─ Components/
│   └─ Fields.razor              ← D1 trin
6
│   └─ Icons.cs                  ← D1 trin
6
│   └─ Recap.razor               ← D1 trin
6
│   └─ StepRail.razor            ← D2 trin
4
├─ Properties/
│   └─ launchSettings.json       (fra
templaten)
└─ wwwroot/
    └─ index.html                ← D1 trin
8
    └─ favicon.png
    └─ css/app.css               ← D1 trin
8, så D2 trin 3-7
    └─ js/app.js                 ← D1 trin
8
```

CSS'en har fire sektioner når du er færdig: MATERIALS , VARIANT 3 – CHECKLIST , APP SHELL og SENT QUOTES . Filer fra prototypen der *ikke* skal med: Proto.razor , Components/Picker.razor , Variants/Stepper.razor , Variants/Conversational.razor , wwwroot/js/picker.js .